Benchmark Presentation — Speaker Script

1. Opening: from one file to thirty-one

Last time we tested our system on one real MEP file. Today we go further. We run our full pipeline over **31 real IFC files**, and we ask one simple question: can our tool really read a building model from start to finish — and where does it honestly stop?

To answer this well, we did not just pick 31 random files. We divided them into three groups on purpose, because each group tests a different ability. Let me go through the three groups now.

2. The three groups in our test set

Group 1 — official examples (12 files). These come from buildingSMART. The same building appears in three IFC versions: IFC2x3, IFC4 and IFC4.3. We use them to test **version stability**. A building should be read in the same way no matter which version it was saved as.

Group 2 — real projects (9 files). These are real buildings made by real software — Archicad, Revit and Allplan. They include a house, an office and an institute, from about 2 MB up to a 62 MB tower. We use them to test **reading in the real world**, with all its mess.

Group 3 — MEP and infrastructure (10 files). This group is different inside. Part of it is MEP, like HVAC, plus one full 28 MB Revit MEP project full of pipes and ducts. The other part is infrastructure — city water pipes, a power network, a bridge. Buildings are divided into floors, but bridges and street pipes are not. So these files have **no storeys**. We put such files in the test on purpose, to see what our tool does when it meets a building-style rule system on a file that is not a building.

So, with these three groups, we test version stability, real-world reading, and honest behaviour on files outside our focus.

3. One hard example: Lab.ifc

Before the batch, let us look at one very hard file to show what "reading" means in practice.

Lab.ifc is an IFC-4 model. The tool finds one storey but zero rooms. Building parts and structure parts are read well. A large set of MEP parts is loaded, and most of their details are complete — but the pipe system names are missing. 82 MEP-structure clashes are found, and 19 of them are checked with real geometry. The file gets an overall health score of 90 out of 100.

Some results can only be worked out when the file has good data and good geometry. When an item cannot be checked, there are two reasons. First, the source model simply does not have the information — for example, the missing pipe system names here. Second, our tool has its own limits — for example, it cannot fully solve clashes on curved shapes. In both cases, our tool reports the problem clearly. It does not make up an answer and does not hide the gap.

4. How we score one model: the metrics slide

Now let me explain the way we score. Every IFC file goes through our pipeline and comes out as **one row of 18 numbers**. We use FZK-Haus as a real example on the slide. The row has three parts.

Part 1 — file information. It records the source, the file name, the IFC version and the file size. This basic data lets others repeat our test later and compare results.

Part 2 — six weighted axes that score our tool.

- **A-robust**: Can the tool open, read and review the model without stopping or failing? (no weight note — part of the six)
- **B-extraction (25%)**: How much of each element type did we read out? It has the highest weight, because any element we miss will stop all later checks on it.
- **C-fieldcov (20%)**: How complete are the element details? Missing details stop rules from judging.
- **D-accuracy (15%)**: Do the numbers we read fall into real engineering ranges? This catches reading mistakes.
- **E-ruleactivation (15%)**: How many of our rules really ran? A low number usually means the input data is incomplete, not that the rules are wrong.
- **F-perf (5%)**: How fast can we read the file, in seconds per megabyte.

The weighted scores give a total score and a grade from S to D.

Part 3 — what this review actually found on this model.

- **qa_score**: The health score of the input IFC file itself.
- **storeys, rooms**: How many floors and rooms were found.
- **issues**: Real rule breaks that we can confirm.

- **not_checked**: A very important number. These are items we could not judge because the source data is missing. We do not invent false problems.
- **parse_secs**: How many seconds the reading took.

To be clear: Part 2 scores **our tool**, while Part 3 tells us about **the quality of the tested file**. In short, one row tells us both how well we worked and how good the model was.

5. What the 31 files showed us

After the batch run, three clear patterns came out — patterns that a single-file test can never show.

First, when the data is complete, our reading is close to perfect. On the real buildings and the MEP project, extraction (B), geometry (D) and speed (F) all stay near 100. The 28 MB Revit project alone gives 2,251 MEP elements, and every type is read. All the real-project models reach grade A or S — FZK-Haus gets 91.0, and the 62 MB tower gets 91.1.

Second, the tool is stable across IFC versions. The same building read from IFC2x3, IFC4 and IFC4.3 gives almost the same scores and results. Many similar tools cannot do this.

Third, the results are layered, not just a long list. On the Revit MEP file, the review raised 1,034 findings, but it separates them honestly. 21 are real reversed-slope pipes (High). 517 are only sent to people for a quick check (Low). More than three hundred are marked as missing data — we say "we cannot know", not a guessed number.

And for the files that truly have no building structure — the bridge, the city pipes, and two very old files from 2005 that use an IFC version even the readers do not fully support — the tool does not fake a result. It reports "no storeys, nothing to review here", gives a low score, and moves on. This honesty is a feature, not a bug.

6. Three key conclusions

Let me close the results with three key points.

First, our reading engine is fully proven. The problem is not our ability to read; it is the data we are given. When the model data is complete, B, D and F are near 100. Every failure comes from missing data, not from our reader. Spending more time on the reader brings less and less benefit.

Second, our main challenge has changed. It is no longer "can we read the file?" It is now "where do we get the data that review needs?" The rule-activation rate is only 32% because IFC files often lack the details rules need, like pipe types. Adding more rules would only widen this data gap.

Third, marking items as "not checked" is our honest strength today — but we must go further. Being honest is good, but if we only say "we cannot check this", users will ask for more. The real value is to help them fill in the missing data and then finish the check. In other words, the next step is to move from "we tell you what is missing" to "we help you complete it and keep reviewing".

7. One-line close

Our tool has learned to read thoroughly. The next job is to feed it — to supply the missing information that the model never recorded, so that our rules stop sitting idle.